
PYTHON PROGRAMMING INTERNSHIP PROJECT REPORT

Submitted in partial fulfilment of the requirements for the
CodeAlpha Python Programming Internship

Projects Undertaken

Stock Portfolio Tracker

Task Automation with Python Scripts (Email Extractor)

Basic Rule-Based Chatbot

Submitted by

Ajay P

CA/DF1/213658

Place: Palladam

Date: 29/07/2026

DECLARATION

I, **Ajay**, hereby declare that this project report titled “**Python Programming Internship Project Report — Stock Portfolio Tracker, Task Automation with Python Scripts, and Basic Rule-Based Chatbot**” is a record of original work carried out by me as part of the CodeAlpha Python Programming Internship.

I further declare that this report, or any part of it, has not been submitted previously for the award of any degree, diploma, or similar title to this or any other institution. All sources of information used in the preparation of this report have been duly acknowledged in the Bibliography section.

Place: Palladam

Date: 29/07/2026

AJAY P

CA/DF1/213658

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **CodeAlpha** for providing me the opportunity to undertake this Python Programming Internship and for their guidance and mentorship throughout its duration.

I am also thankful to **[Internal Faculty Guide Name]**, my internal project guide, and **[Head of Department Name]**, Head of the **[Department Name]**, for their constant support, encouragement, and valuable suggestions during the course of this project.

I extend my thanks to my family and friends for their continuous encouragement, and to everyone who directly or indirectly contributed to the successful completion of this internship and this report.

AJAY P

TABLE OF CONTENTS

TOC_PLACEHOLDER_START - CERTIFICATE - DECLARATION -
ACKNOWLEDGEMENT - ABSTRACT - INTRODUCTION - OBJECTIVES OF THE
PROJECT - TOOLS AND TECHNOLOGIES USED - SYSTEM REQUIREMENTS - TASK
2: STOCK PORTFOLIO TRACKER - TASK 3: TASK AUTOMATION WITH PYTHON
SCRIPTS - TASK 4: BASIC RULE-BASED CHATBOT - TESTING - RESULTS AND
DISCUSSION - CONCLUSION - FUTURE SCOPE - BIBLIOGRAPHY
TOC_PLACEHOLDER_END

ABSTRACT

This report documents the design, implementation, and testing of three Python programming mini-projects completed as part of the CodeAlpha Python Programming Internship: a **Stock Portfolio Tracker**, a **Task Automation script (email extractor)**, and a **Basic Rule-Based Chatbot**. Each project applies core Python concepts — dictionaries, lists, loops, conditional branching, string formatting, functions, file handling, and regular expressions — to a small, self-contained, real-world-style problem.

For each task, this report presents the problem definition, the methodology followed, the complete source code, and console output captured from actual program execution. During implementation review, two logic defects were identified in the Stock Portfolio Tracker and one functional defect was identified in the email-extraction automation script; both are documented with root-cause analysis, corrected, and re-verified against real test data. The chatbot was found to function correctly as designed. The report concludes with a discussion of results, testing performed, and directions for future enhancement of each project.

INTRODUCTION

About CodeAlpha

CodeAlpha is a software development company that offers structured, hands-on internship programs in domains such as Python programming, web development, data science, and Android development. The Python Programming track is designed to give interns practical experience with Python fundamentals, data structures, file handling, and object-oriented concepts, while working toward a portfolio of real, runnable mini-projects.

About the Internship

The internship requires interns to complete a minimum of two to three tasks out of four offered in their chosen domain in order to be eligible for a certificate of completion. For the Python Programming domain, the four available tasks are: a Hangman game, a Stock Portfolio Tracker, a Task Automation script, and a Basic Chatbot. Interns are additionally expected to publish their source code to GitHub, share a short project walkthrough on LinkedIn, and submit their work through an official submission form.

Scope of This Report

This report covers **three** of the four available Python tasks — the **Stock Portfolio Tracker**, **Task Automation with Python Scripts**, and the **Basic Chatbot** — satisfying the internship's minimum completion requirement. Each task is treated as an independent mini-project with its own problem statement, implementation, and results, and is presented in a uniform structure for ease of evaluation.

OBJECTIVES OF THE PROJECT

The primary objectives of undertaking these three projects were:

1. To apply core Python programming constructs — dictionaries, lists, loops, conditionals, functions, and string formatting — to practical, self-contained problems.
2. To gain hands-on experience with **file handling** in Python, including reading from and writing to text files.
3. To gain hands-on experience with **regular expressions** for pattern-based text extraction.
4. To practice building a simple **rule-based decision system** (the chatbot) using conditional branching.
5. To develop the ability to **trace, test, debug, and correct** logic errors in existing code — a core software engineering skill exercised while reviewing the Stock Portfolio Tracker and the automation script.
6. To produce clean, readable, well-documented Python code suitable for submission to a public GitHub repository, in line with the internship's submission guidelines.

TOOLS AND TECHNOLOGIES USED

Category	Tool / Technology
Programming Language	Python 3
Core Modules Used	Built-in dict, list, str; the re module (regular expressions)
Development Environment	Any standard Python IDE / text editor (e.g. VS Code, PyCharm, IDLE)
Version Control	Git & GitHub (for source code submission, per internship guidelines)
Documentation	Markdown / PDF (this report)
Operating System	Cross-platform (Windows / macOS / Linux) — no OS-specific code used

No external, third-party Python packages were required for any of the three tasks — all three programs run using only the Python standard library.

SYSTEM REQUIREMENTS

Hardware Requirements

- A standard personal computer or laptop
- Minimum 4 GB RAM (no significant computational load)
- Minimal disk space (each script and its output files total a few kilobytes)

Software Requirements

- Python 3.6 or later (f-strings, used throughout, require Python ≥ 3.6)
- A text editor or IDE capable of running files
- No internet connection or external API access is required by any of the three submitted scripts

STOCK PORTFOLIO TRACKER

1 Problem Definition

To build a console-based program that calculates an investor's total investment across a set of stocks. Stock prices are stored in a hardcoded dictionary. The user enters the number of stocks to record, then for each one enters the ticker symbol and quantity purchased. The program must display a formatted summary table and optionally save the report to a text file.

1.1 Methodology / Algorithm

1. Define a dictionary stock prices mapping ticker symbols to unit prices.
2. Prompt the user for the number of stocks n .
3. Repeat n times: read a ticker symbol, validate it against the dictionary, read the quantity purchased, compute $\text{price} \times \text{quantity}$, and store the result.
4. Print a formatted table of all recorded stocks.
5. Print the total investment across the whole portfolio.
6. Write the same report to portfolio.txt.

1.2 Source Code

```
stock_prices = {
    "AAPL": 180,
    "TSLA": 250,
    "GOOGL": 140,
    "MSFT": 330,
    "AMZN": 135
}

investment = 0
portfolio = []
n = int(input("Enter the number of stocks: "))
for i in range(n):
    stock = input("\nEnter stock name (AAPL, TSLA, GOOGL, MSFT, AMZN): ").upper()
    if stock in stock_prices:
        quantity = int(input("Enter quantity: "))
        investment = stock_prices[stock] * quantity
        investment += investment
        portfolio.append([stock, quantity, stock_prices[stock], investment])
    else:
        print("Stock not found!")
print("\n----- STOCK PORTFOLIO -----")
print("{:<10} {:<10} {:<12} {:<15}".format("Stock", "Qty", "Price", "Investment"))
```

```

for item in portfolio:
    print("{:<10} {:<10} {:<12} ${:<15}".format(item[0], item[1], item[2], item[3]))

print("\nTotal Investment = $", investment)
file=open("portfolio.txt", "w")
file.write("Stock Portfolio\n")
file.write("-----\n")
for item in portfolio:
    file.write(f'{item[0]} Qty:{item[1]} Price:${item[2]} Investment:${item[3]}\n')
file.write(f'\nTotal Investment = ${investment}')
print("\nPortfolio saved successfully to 'portfolio.txt'")

```

1.3 Sample Output

Enter the number of stocks: 3
 Enter stock name (AAPL, TSLA, GOOGL, MSFT, AMZN): AAPL
 Enter quantity: 4
 Enter stock name (AAPL, TSLA, GOOGL, MSFT, AMZN): tsla
 Enter quantity: 5
 Enter stock name (AAPL, TSLA, GOOGL, MSFT, AMZN): googl
 Enter quantity: 6

```

----- STOCK PORTFOLIO -----
Stock  Qty    Price   Investment
AAPL   4      180     $1440
TSLA   5      250     $2500
GOOGL  6      140     $1680

```

Total Investment = \$ 1680
 Portfolio saved successfully to 'portfolio.txt'

1.4 Result and Bug Analysis

Tracing the program against the sample input reveals two logic defects:

Doubled investment values. The line `investment += investment` immediately follows the correct calculation `investment = stock_prices[stock] * quantity`, doubling it before storage (e.g. AAPL's true cost of \$1800 is recorded and displayed as \$3600).

Incorrect grand total. `investment` is overwritten each loop pass and never accumulated, so the printed "Total Investment" (\$1320) is only the last stock's amount, not the sum of the portfolio (which should be \$7420 using the doubled figures, or \$3710 using the correct, non-doubled figures).

TASK AUTOMATION WITH PYTHON SCRIPTS

2 Problem Definition

To automate a small, repetitive real-life task using Python. The variant implemented here is to extract every email address present in a block of text and save the results to a separate output file, using a regular expression to identify valid email-shaped substrings.

2.1 Methodology / Algorithm

1. Open and read the contents of a source text file.
2. Define a regular-expression pattern that matches the general shape of an email address (local-part @ domain . TLD)
3. Apply the pattern against the source text using `re.findall()`
4. Write each matched address to an output file.
5. Report the total number of addresses found.

2.2 Source Code

```
emails = (r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}")
file=open("emails.txt","w")
for email in emails:
    file.write(email)
file = open("emails.txt","r")
file.read()
print("Email addresses extracted successfully!")
print("Total Emails Found:", len(emails))
```

2.1 Sample Output

Email addresses extracted successfully!
Total Emails Found: 46

Generated file — `emiala.txt`
`emails = (r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}")`

2.2 Sample Output

Email addresses extracted successfully!
Total Emails Found: 46

Generated file — `emaila.txt`

```
emails = (r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}")
```

2.3 Result and Defect Analysis

Tracing the code shows the program does not perform email extraction at all:

- `email` is assigned the **regex pattern itself**, as a plain 46-character string — it is never the *result* of a search.
- For `email` in `emails` iterates over that string one character at a time (Python strings are iterable by character), so the pattern text is copied into character.
- `len(emails)` reports the **length of the pattern string** (46), not a count of matched addresses —
no source document is ever opened, and the `re` module's matching functions (`re.findall` / `re.search`) are never invoked, despite the pattern being written in valid regex syntax.

2.5 Source file used for testing — `sample_data.txt`:

Hi team,
Please reach out to john.doe@example.com for project queries, or contact the HR department at hr.support@codealpha.tech.
For billing issues email billing@company.co.in.
You can also cc jane_smith99@gmail.com on urgent matters.
Thanks,
Admin

2.6 Actual output of the corrected script:

```
john.doe@example.com  
hr.support@codealpha.tech  
billing@company.co.in  
jane_smith99@gmail.com
```

Email addresses extracted successfully!
Total Emails Found: 4

`len(emails)` now correctly reports 4, matching the four genuine addresses present in the source file, confirming the corrected script performs true pattern-based extraction

BASIC RULE-BASED CHATBOT

3 Problem Definition

To build a simple, rule-based, text-based chatbot that recognizes a small, fixed set of user phrases (such as "hello", "how are you", "bye") and responds with a predefined reply for each, continuing the conversation until the user signals they wish to end it.

3.1 Methodology / Algorithm

1. Print a welcome banner explaining how to exit.
2. Repeatedly read one line of user input, normalized to lowercase.
3. Compare the normalized input against each recognized phrase using an if / elif chain; print the matching canned reply.
4. If the input matches none of the recognized phrases, print a generic "I don't understand" fallback reply.
5. If the input is "bye", print a farewell message and exit the loop.

3.2 Source Code

```
def chatbot():
    print(" Welcome to the Basic Chatbot ")
    print("Type 'bye' to end the chat.\n")
    while True:
        user = input("You: ").lower()
        if user == "hello":
            print("Bot: Hi! How can I help you?")
        elif user == "hi":
            print("Bot: Hello! Nice to meet you.")
        elif user == "how are you":
            print("Bot: I'm fine, thanks! How about you?")
        elif user == "i am fine":
            print("Bot: That's great to hear!")
        elif user == "what is your name":
            print("Bot: I am a simple Python chatbot.")
        elif user == "who created you":
            print("Bot: I was created using Python.")
        elif user == "thank you":
            print("Bot: You're welcome!")
        elif user == "bye":
            print("Bot: Goodbye! Have a nice day.")
            break
        else:
            print("Bot: Sorry, I don't understand that.")
# Run the chatbot
chatbot()
```

3.3 Sample Output

Conversation: hello -> how are you -> I am fine -> what is your name -> who created -> thank you -> bye

Welcome to the Basic Chatbot Type
'bye' to end the chat.
You: hello
Bot: Hi! How can I help you?
You: how are you
Bot: I'm fine, thanks! How about you?
You: i am fine
Bot: That's great to hear! You:
what is your name
Bot: I am a simple Python chatbot.
You: who created you
Bot: I was created using Python.
You: thank you
Bot: You're welcome!
You: bye
Bot: Goodbye! Have a nice day.

Unrecognized-input test: Hi → what is 2+2 → bye

3.4 Result

The chatbot performs correctly against every phrase in its rule set and correctly falls back to the generic reply for unrecognized input, with no functional defects found during testing. Two minor robustness improvements are recommended for future revisions: (a) matching phrases using substring containment ("hello" in user) rather than strict equality, so that variations like "hello there" are still recognized, and (b) guarding the top-level chatbot() call with `if __name__ == "__main__":` so the chat does not start automatically if the file is ever imported as a module.

TESTING

Each of the three programs was tested by actually executing the script with representative, edge-case, and invalid inputs, and comparing the observed output against the expected result computed independently.

Task	Test Case	Input	Expected Result	Actual Result	Status
Stock Portfolio Tracker	Valid multi-stock entry	AAPL×10, TSLA×5, MSFT×2	Total = \$3710	\$1320 (last item only, doubled)	Fail — defect found & fixed
Stock Portfolio Tracker	Corrected version, same input	AAPL×10, TSLA×5, MSFT×2	Total = \$3710	\$3710	Pass
Stock Portfolio Tracker	Corrected version, other tickers	GOOGL×8, AMZN×4	Total = \$1660	\$1660	Pass
Stock Portfolio Tracker	Invalid ticker	XYZ then AAPL×3	“Stock not found!” printed, only AAPL recorded	“Stock not found!” printed, only AAPL recorded	Pass
Task Automation with Python Scripts.	Submitted script	(no source file)	Real emails extracted	Regex pattern text copied into output file	Fail — defect found & fixed
Task Automation with Python Scripts.	Corrected script, sample file 1	4-email text block	4 emails extracted	4 emails extracted correctly	Pass
Task Automation with Python Scripts.	Corrected script, sample file 2 (with invalid entry)	3 valid + 1 invalid string	3 emails extracted, invalid entry ignored	3 emails extracted, invalid entry ignored	Pass
Basic chatbot	Recognized phrases	Full 7-phrase conversation	All correct canned replies	All correct canned replies	Pass
Basic chatbot	Unrecognized input	"what is 2+2"	Fallback reply printed	Fallback reply printed	Pass
Basic chatbot	Exit condition	"bye"	Farewell printed, loop exits	Farewell printed, loop exits	Pass

RESULTS AND DISCUSSION

Of the three tasks reviewed, two (**Stock Portfolio Tracker** and **Task Automation Script**) contained logic or functional defects in their originally submitted form, while one (**Basic Chatbot**) was fully correct as submitted.

Stock Portfolio Tracker : correctly demonstrated the required concepts — dictionary lookups, loops, formatted output, and file writing — but a stray self-doubling line and a missing accumulator variable caused every reported investment figure to be incorrect. Both defects were traced to their exact source line, corrected, and the fix was independently verified against two different sets of test inputs.

Task Automation Script : demonstrated familiarity with regular-expression syntax but never invoked the `re` module's matching functions against real text, meaning the script's stated purpose — extracting email addresses — was never actually achieved. A corrected version was implemented, and verified against two independent sample text files, one of which deliberately included a non-email string to confirm the pattern correctly rejects invalid entries.

Basic Chatbot : met its specification with no functional defects. Only optional robustness improvements (fuzzy phrase matching, a `__main__` guard) were suggested as future enhancements rather than corrections.

Overall, this exercise reinforced the importance of manually tracing program logic against realistic sample data — rather than assuming a program is correct simply because it runs without raising an error — since both defective scripts executed to completion without any runtime exceptions, despite producing incorrect results

CONCLUSION

This project successfully implements and documents three independent Python mini-projects — a Stock Portfolio Tracker, an email-address extraction automation script, and a rule-based chatbot — completed as part of the CodeAlpha Python Programming Internship. Each project exercises a distinct set of core Python skills: dictionary-based data lookup and file output for the portfolio tracker; regular-expression-based text pattern matching for the automation script; and function design with conditional branching and loop control for the chatbot.

During the course of implementation review, real logic defects were identified in two of the three scripts, root-caused, corrected, and re-verified using actual program execution rather than assumption a process that mirrors real-world software debugging practice. All three corrected programs now behave as intended and are ready for submission to individual GitHub repositories, satisfying CodeAlpha's requirement of completing a minimum of two to three tasks for internship certification.

FUTURE SCOPE

- **Stock Portfolio Tracker** : integrate a live stock-price API (e.g. via the requests library) in place of the hardcoded dictionary, add CSV export, and support merging repeated purchases of the same ticker.
- **Task Automation Script** : extend the automation to also support the other two CodeAlpha-listed variants — moving .jpg files between folders using os / shutil , and scraping a fixed webpage's title using requests — so that a single automation toolkit covers all three ideas from the task brief.
- **Basic Chatbot** : replace the fixed if/elif phrase list with a dictionary-driven or lightweight NLP based intent matcher, add conversation logging to a file, and introduce varied, randomly-selected replies for a less repetitive user experience.

BIBLIOGRAPHY

1. Python Software Foundation, The Python Standard Library – re module (Regular Expression Operations), docs.python.org
 2. Python Software Foundation, Built-in Types – dict , list , str , docs.python.org
 3. Python Software Foundation, Input and Output – Reading and Writing Files, docs.python.org
 4. Python Software Foundation, string – Common String Operations – Format String Syntax, docs.python.org
 5. CodeAlpha, Python Programming Tasks & Instructions, codealpha.tech – internship task brief referenced throughout this report
-